

Lab 1:

The screenshot shows the pgAdmin interface with a SQL query window. The query is as follows:

```
1 EXPLAIN (ANALYZE, BUFFERS)
2 SELECT c.first_name, c.last_name, r.rental_date
3 FROM customer c
4 JOIN rental r ON c.customer_id = r.customer_id
5 WHERE c.customer_id = 1;
6
```

Below the query window, the 'Data Output' tab is active, displaying the 'QUERY PLAN' for the executed query. The plan shows a 'Nested Loop' join, which is efficient in this case because the 'customer' table is filtered to a single row by the 'WHERE' clause. The 'rental' table is then scanned for rows matching the single customer.

Step	Operation	Cost	Rows	Width	Actual Time	Actual Rows	Loops
1	Nested Loop	(cost=0.28..359.16 rows=32 width=21)			(actual time=1.784..15.819 rows=33.00 loops=1)		
2	Buffers: shared hit=153 dirtied=1						
3	-> Index Scan using customer_pkey on customer c	(cost=0.28..8.29 rows=1 width=17)			(actual time=1.320..1.324 rows=1.00 loops=1)		
4	Index Cond: (customer_id = 1)						
5	Index Searches: 1						
6	Buffers: shared hit=3						
7	-> Seq Scan on rental r	(cost=0.00..350.55 rows=32 width=10)			(actual time=0.457..14.471 rows=33.00 loops=1)		

Total rows: 14 | Query complete 00:00:00.117

- **Identify the Roles :** The customer table is the Outer table, while the rental table is the Inner table. PostgreSQL first finds the customer with `customer_id = 1`, then looks up matching rental records.
- **Why Nested Loop?:** The customer table returns only 1 row because `customer_id` is the primary key and the query filters on `customer_id = 1`. A Nested Loop Join is efficient in this situation because PostgreSQL only needs to perform the inner lookup for one customer. A Hash Join would introduce unnecessary overhead by building a hash table.
- **Efficiency:** The Inner rental node has `loops = 1` because the Outer customer node produces only one row. Therefore, PostgreSQL executes the inner lookup only once to find all rental records belonging to customer 1.

Lab 2:

The screenshot shows a PostgreSQL query editor with a query window and a query plan window. The query is:

```
1 EXPLAIN (ANALYZE, BUFFERS)
2 SELECT f.title, l.name AS language
3 FROM film f
4 JOIN language l ON f.language_id = l.language_id;
```

The query plan window shows the following details:

Step	Operation	Cost	Time	Rows	Width	Loops
1	Hash Join	(cost=1.14..103.62 rows=1000 width=99)	(actual time=0.847..7.222 rows=1000.00 loops=1)			
2	Hash Cond: (f.language_id = l.language_id)					
3	Buffers: shared hit=89 dirtied=4					
4	-> Seq Scan on film f	(cost=0.00..98.00 rows=1000 width=17)	(actual time=0.497..6.488 rows=1000.00 loops=1)			
5	Buffers: shared hit=88 dirtied=4					
6	-> Hash	(cost=1.06..1.06 rows=6 width=88)	(actual time=0.311..0.312 rows=6.00 loops=1)			
7	Buckets: 1024 Batches: 1 Memory Usage: 9kB					

Total rows: 14 Query complete 00:00:00.124

Successfully run. Total query runtime: 124 msec. 14 rows affected.

- **Identify Phases:** The language table is hashed because it is the smaller table. PostgreSQL performs a sequential scan on language and builds a hash table using language_id. The film table is then scanned and used to probe the hash table using film.language_id.
- **Memory Check:** The Hash node uses approximately 9 kB of memory in the typical dvdrental execution plan. This memory is used to store the hash table built from the language table. The actual value should be taken from the Memory Usage line in the execution plan.

The screenshot shows a PostgreSQL query editor with a query window and a query plan window. The query is:

```
1 SET work_mem = '64kB';
2 EXPLAIN (ANALYZE, BUFFERS)
3 SELECT f.title, l.name AS language
4 FROM film f
5 JOIN language l ON f.language_id = l.language_id;
```

The query plan window shows the following details:

Step	Operation	Cost	Time	Rows	Width	Loops
1	Hash Join	(cost=1.14..103.62 rows=1000 width=99)	(actual time=1.396..2.051 rows=1000.00 loops=1)			
2	Hash Cond: (f.language_id = l.language_id)					
3	Buffers: shared hit=89					
4	-> Seq Scan on film f	(cost=0.00..98.00 rows=1000 width=17)	(actual time=0.031..0.321 rows=1000.00 loops=1)			
5	Buffers: shared hit=88					
6	-> Hash	(cost=1.06..1.06 rows=6 width=88)	(actual time=0.049..0.050 rows=6.00 loops=1)			
7	Buckets: 1024 Batches: 1 Memory Usage: 9kB					

Total rows: 12 Query complete 00:00:00.075

Successfully run. Total query runtime: 75 msec. 12 rows affected.

- **Hypothesis:** After setting work_mem to 64kB, the Hash Join can still execute normally because the language table is very small. Its hash table fits within the available memory, so the Hash node normally remains at Batches: 1. A larger hash table could require multiple batches and potentially disk-based processing when insufficient memory is available.

Lab 3:

The screenshot shows a PostgreSQL IDE interface. The top panel displays a SQL query:

```

1 SET enable_hashjoin = OFF;
2 SET enable_nestloop = OFF;
3 EXPLAIN (ANALYZE, BUFFERS)
4 SELECT f.film_id, f.title, i.inventory_id
5 FROM film f
6 JOIN inventory i ON f.film_id = i.film_id
7 ORDER BY f.film_id;

```

The bottom panel shows the query plan:

```

QUERY PLAN
text
1 Merge Join (cost=412.64..622.36 rows=4581 width=23) (actual time=21.497..25.341 rows=4581 loops=1)
2 Merge Cond: (f.film_id = i.film_id)
3 Buffers: shared hit=95 read=23 dirtied=2, temp read=10 written=12
4 -> Index Scan using film_pkey on film f (cost=0.28..127.33 rows=1000 width=19) (actual time=0.011..1.006 rows=1000 loops=1)
5 Index Searches: 1
6 Buffers: shared hit=89 read=1 dirtied=1
7 -> Materialize (cost=412.37..435.27 rows=4581 width=6) (actual time=21.477..22.899 rows=4581 loops=1)
Total rows: 19 Query complete 00:00:00.130

```

A green status bar at the bottom right indicates: "Successfully run. Total query runtime: 130 msec. 19 rows affected."

- **Structure:** The execution plan uses a Merge Join. PostgreSQL needs both inputs to be ordered by film_id. Depending on the available indexes, the plan may use an Index Scan to obtain sorted data directly or use a Sort node before the join. The inventory table can use its index on film_id to provide ordered rows.

The screenshot shows a PostgreSQL query editor with the following SQL query:

```

1 SET enable_hashjoin = OFF;
2 SET enable_nestloop = OFF;
3 EXPLAIN (ANALYZE, BUFFERS)
4 SELECT f.film_id, f.title, i.inventory_id
5 FROM film f
6 JOIN inventory i ON f.film_id = i.film_id
7 ORDER BY f.film_id;
8 RESET ALL;
9 EXPLAIN (ANALYZE, BUFFERS)
10 SELECT f.film_id, f.title, i.inventory_id
11 FROM film f
12 JOIN inventory i ON f.film_id = i.film_id
13 ORDER BY f.film_id;

```

The query plan shows a Sort operation followed by a Hash Join. The Hash Join cost is 110.50, and the Sort cost is 471.95. The query was successfully run, affecting 18 rows in 106 msec.

- **Compare:** After running RESET ALL, Hash Join and Nested Loop are enabled again. PostgreSQL can therefore choose the join method based on its cost estimates. For this query, the optimizer may choose a Hash Join because the join condition is an equality condition and building a hash table can be cheaper than sorting the inputs for a Merge Join.

Hands-on Lab: Optimizing dvdrental:

Task 1: Influence the Optimizer

The screenshot shows a PostgreSQL query editor with the following SQL query:

```

1 SET enable_indexscan = off;
2
3 EXPLAIN ANALYZE
4 SELECT *
5 FROM rental
6 WHERE rental_date > '2005-05-24';

```

The query plan shows a Seq Scan on rental with a filter on rental_date. The query was successfully run, affecting 7 rows in 123 msec.

Query History

```

1 SET enable_indexscan = off;
2
3 EXPLAIN ANALYZE
4 SELECT *
5 FROM rental
6 WHERE rental_date > '2005-05-24';
7 SET enable_indexscan = on;
8
9 EXPLAIN ANALYZE
10 SELECT *
11 FROM rental
12 WHERE rental_date > '2005-05-24';

```

Data Output Messages Notifications

Showing rows: 1 to 7 Page No: 1 of 1

QUERY PLAN

```

text
1 Seq Scan on rental (cost=0.00..350.55 rows=16044 width=36) (actual time=0.019..1.448 rows=16045.00 loop=1)
2   Filter: (rental_date > '2005-05-24 00:00:00':timestamp without time zone)
3   Buffers: shared hit=150
4   Planning:
5   Buffers: shared hit=4
6   Planning Time: 0.252 ms
7   Execution Time: 1.935 ms

```

Successfully run. Total query runtime: 139 msec. 7 rows affected.

Task 2 :

Query History

```

1 EXPLAIN
2 SELECT c.customer_id,
3        c.first_name,
4        c.last_name,
5        r.rental_id,
6        r.rental_date,
7        i.inventory_id
8 FROM customer c
9 JOIN rental r
10  ON c.customer_id = r.customer_id
11 JOIN inventory i
12  ON r.inventory_id = i.inventory_id;

```

Data Output Messages Notifications

Showing rows: 1 to 9 Page No: 1 of 1

QUERY PLAN

```

text
1 Hash Join (cost=150.55..545.57 rows=16044 width=33)
2   Hash Cond: (r.inventory_id = i.inventory_id)
3   -> Hash Join (cost=22.48..375.33 rows=16044 width=33)
4     Hash Cond: (r.customer_id = c.customer_id)
5     -> Seq Scan on rental r (cost=0.00..310.44 rows=16044 width=18)
6     -> Hash (cost=14.99..14.99 rows=599 width=17)
7       -> Seq Scan on customer c (cost=0.00..14.99 rows=599 width=17)

```

Successfully run. Total query runtime: 116 msec. 9 rows affected.

The screenshot shows a SQL IDE interface. The top toolbar contains icons for file operations, query execution, and help. The main editor displays a SQL query:

```

11 JOIN inventory i
12     ON r.inventory_id = i.inventory_id;
13 ANALYZE customer;
14 ANALYZE rental;
15 ANALYZE inventory;
16 EXPLAIN
17 SELECT c.customer_id,
18        c.first_name,
19        c.last_name,
20        r.rental_id,
21        r.rental_date,
22        i.inventory_id
23 FROM customer c
24 JOIN rental r
25     ON c.customer_id = r.customer_id
26 JOIN inventory i
27     ON r.inventory_id = i.inventory_id;
  
```

Below the query editor, the 'Data Output' tab is active, showing the 'QUERY PLAN' for the query. The plan consists of several steps:

- 1 Hash Join (cost=150.60..545.63 rows=16045 width=33)
- 2 Hash Cond: (r.inventory_id = i.inventory_id)
- 3 -> Hash Join (cost=22.52..375.39 rows=16045 width=33)
- 4 Hash Cond: (r.customer_id = c.customer_id)
- 5 -> Seq Scan on rental r (cost=0.00..310.45 rows=16045 width=18)
- 6 -> Hash (cost=15.01..15.01 rows=601 width=17)
- 7 -> Seq Scan on customer c (cost=0.00..15.01 rows=601 width=17)

The bottom status bar indicates: 'Total rows: 9', 'Query complete 00:00:00.097', and a green message: 'Successfully run. Total query runtime: 97 msec. 9 rows affected.' The right side of the status bar shows 'CRLF' and 'Ln 27, Col 40'.